Practice Lab:

DSE Core (and more), Security

- This Practice Lab is dependent on Discussion Unit 6220, where most of the objects we create in this lab were introduced.
- In this Practice Lab, we DSE initiate (internal, native) Authentication, and apply Authorization rules to a number of database objects.

Challenge 1: Enable/Test DSE Security

Prerequisites:

- Instructions are provided for a single node DSE Core cluster.
- *There are additional steps for a multi-node cluster, that we overview only.*

- Instructions are provided for the command prompt using CQLSH.

- All work done as ‘root’

Challenge 1: DSE Security, In the Real World ..

- Implementing Authorization/Authentication can be done without downtime
  - -- Clients: Roll out new client programs with TRANSITIONAL client side driver switch (details on Notes page)
  - -- Server: change in multi-user mode, or rolling change
- Multi-node:
  - -- ALTER KEYSPACE, system_auth, dse_security
  - -- nodetool repair

DSE Security: Sample Artifacts to Secure


```
DROP KEYSPACE IF EXISTS ks_6221;

CREATE KEYSPACE ks_6221
   WITH REPLICATION =
   {'class': 'SimpleStrategy',
   'replication_factor': 1};

USE ks_6221;
```

DSE Security: Sample Artifacts to Secure

```
DROP TABLE IF EXISTS cust_orders;

CREATE TABLE cust_orders
   (
   region              TEXT,
   cust_name           TEXT,
   ord_num             INT,
   other               TEXT,
   PRIMARY KEY ((region, cust_name),
      ord_num)
   );
```

DSE Security: Sample Artifacts to Secure

```
INSERT INTO cust_orders
   (region, cust_name, ord_num, other)
VALUES ('EMEA', 'IKEA'   , 101, 'Shoes'         );
   ...
VALUES ('NA'   , 'SEARS' , 101, 'Shoes, Washer');
VALUES ('NA'   , 'SEARS' , 102, 'Oranges'       );
VALUES ('NA'   , 'MACYS' , 101, 'Dress, Tie'    );
```

DSE Security: Sample Artifacts to Secure

```
DROP TABLE IF EXISTS cust_payments;

CREATE TABLE cust_payments
   (
   cust_name          TEXT,
   payment_num        INT,
   other              TEXT,
   PRIMARY KEY ((cust_name, payment_num))
   );
INSERT INTO cust_payments (cust_name,
   payment_num, other)
VALUES ('SEARS' , 101, '$100');
   ...
VALUES ('MACYS' , 101, '$200');
```

DSE Security: cassandra.yaml

Confirm or create these changes-

```
# default, confirm set
authenticator: com.datastax.bdp.cassandra.auth.DseAuthenticator

# default, confirm set
role_manager: com.datastax.bdp.cassandra.auth.DseRoleManager

# If doing RLAC (Yes, you are), tune
#
#  Uncomment any that are commented
#  permissions_cache_max_entries  not present in 6.0
permissions_validity_in_ms: 2000
permissions_uddate_interval_in_ms: 2000
permissions_cache_max_entries: 1000
```

DSE Security: dse.yaml

Confirm or create these changes-

```
#  Uncomment all
#  Set,      enabled: true
#  Ensure,   default_scheme: internal
authentication_options:
   enabled: true
   default_scheme: internal
   other_schemes:
   scheme_permissions: false
   allow_digest_with_kerberos: true
   plain_text_without_ssl: warn
   transitional_mode: disabled
#  Uncomment
role_management_options:
   mode: internal
#  Uncomment
#  Set,      enabled: true
authorization_options:
   enabled: true
   transitional_mode: disabled
   allow_row_level_security: true
```

DSE Security: then reboot DSE

Again; not required when in production. Easier when testing.

- Less steps
- Error messages on boot screen/log

DSE Security: CREATE ROLE


```
DROP ROLE IF EXISTS bob            ;
DROP ROLE IF EXISTS nancy          ;
DROP ROLE IF EXISTS dirk           ;

DROP ROLE IF EXISTS senior_operator;
DROP ROLE IF EXISTS operator       ;
DROP ROLE IF EXISTS operator_na    ;

CREATE ROLE bob   WITH LOGIN = true AND PASSWORD = 'password';
GRANT EXECUTE on INTERNAL SCHEME to bob  ;
CREATE ROLE nancy WITH LOGIN = true AND PASSWORD = 'password';
GRANT EXECUTE on INTERNAL SCHEME to nancy;
CREATE ROLE dirk  WITH LOGIN = true AND PASSWORD = 'password';
GRANT EXECUTE on INTERNAL SCHEME to dirk ;
```

DSE Security: Assign/CREATE Roles


```
CREATE ROLE senior_operator;

// INSERT, UPDATE, DELETE and TRUNCATE rows in
// any table in the specified keyspace.
// ADD SELECT
GRANT MODIFY, SELECT ON KEYSPACE ks_6221 TO senior_operator;
GRANT senior_operator to nancy;

CREATE ROLE operator;

GRANT MODIFY,SELECT ON TABLE ks_6221.cust_orders   TO operator;
GRANT SELECT        ON TABLE ks_6221.cust_payments TO operator;

GRANT operator to bob;
```

DSE Security: Review of data/other

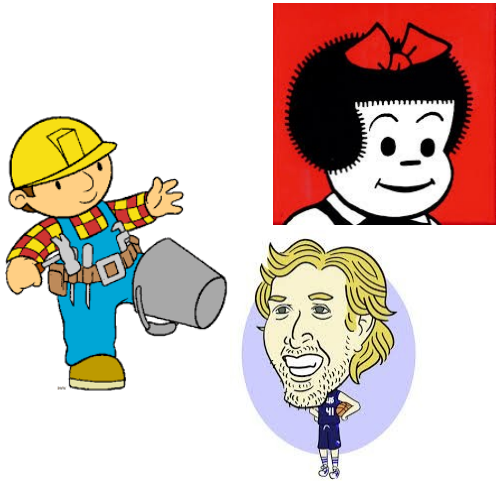
```
CREATE TABLE cust_orders
   (
   region            TEXT,
   cust_name         TEXT,
   ord_num           INT,
   other             TEXT,
   PRIMARY KEY ((region, cust_name), ord_num)
INSERT INTO cust_orders ...
   VALUES ('EMEA', 'IKEA'  , 101, 'Shoes'         );
   VALUES ('NA'  , 'SEARS' , 101, 'Shoes, Washer' );
   VALUES ('NA'  , 'SEARS' , 102, 'Oranges'       );
   VALUES ('NA'  , 'MACYS' , 101, 'Dress, Tie'    );
INSERT INTO cust_payments ...
   VALUES ('SEARS' , 101, '$100'  );
   VALUES ('MACYS' , 101, '$200'  );
```

DSE Security: RLAC

```
CREATE ROLE operator_na;

// RESTRICT ROWS ON ks_6221.cust_orders USING other;
// InvalidRequest: Error from server: code=2200 [Invalid query]
// message="Restrict Rows Statement must be for a Primary Key
// or a Partition Key column"

RESTRICT ROWS ON ks_6221.cust_orders USING region;
RESTRICT ROWS ON ks_6221.cust_orders USING cust_name;
//  DESCRIBE TABLE shows cust_name only

RESTRICT ROWS ON ks_6221.cust_orders USING region;

GRANT SELECT ON 'NA' ROWS IN ks_6221.cust_orders TO operator_na;
GRANT operator_na to dirk;
```

DSE Security: Testing

```
cqlsh -u nancy -p password
cqlsh -u bob   -p password
cqlsh -u dirk  -p password

   use ks_6221;

   select * from cust_orders;
   select * from cust_payments;
```



Nancy, copyright Ernie Bushmiller
Bob the Builder, copyright, Keith Chapman
Dirk Nowitzki, by Chris Sebastian

Practice Lab:

Lessons Learned